

DOCUMENTO TÉCNICO

RESTAURANTEPRO — SISTEMA DE GESTÃO E OPERAÇÃO DE RESTAURANTE

Especificação funcional, técnica, arquitetura, base de dados, fluxos, permissões e critérios de aceitação para entrega ao programador

Documento	Especificação Técnica e Funcional
Versão	1.0
Data	Setembro de 2026
Sistema	RestaurantePro
Ambiente alvo	XAMPP / PHP / MySQL
Idioma	Português
Moeda base	Kz (configurável)
Estado	Documento para desenvolvimento

Objetivo: transformar a referência visual fornecida num sistema operacional real para restaurante, com dados persistentes, regras de negócio, POS, mesas, cozinha, stock, compras, caixa, faturação, relatórios, utilizadores e auditoria. A imagem de referência define principalmente a linguagem visual e a organização do dashboard; não deve ser tratada como especificação completa das regras internas.

1. OBJETIVO E ESCOPO

O sistema deverá centralizar a operação do restaurante numa única aplicação web, permitindo controlar o ciclo completo: abertura de turno → ocupação de mesas → pedido → produção na cozinha → entrega → pagamento → faturação → movimento de caixa → atualização de stock → relatórios.

- Aplicação web responsiva para computador, tablet e ecrãs de operação.
- Arquitetura preparada para uma ou várias filiais/unidades.
- Separação entre interface, regras de negócio, acesso a dados e serviços.
- Todas as operações críticas devem ficar registadas para auditoria.
- Valores, impostos, métodos de pagamento, séries documentais e parâmetros devem ser configuráveis.

2. STACK E ARQUITETURA

2.1 Stack recomendada

- Backend: PHP 8.x, arquitetura MVC/estruturada, sem dependência obrigatória de framework pesado.
- Base de dados: MySQL 8.x/MariaDB compatível com XAMPP.
- Frontend: HTML5, CSS3 e JavaScript modular.
- Comunicação assíncrona: Fetch/AJAX com respostas JSON para operações de POS e cozinha.
- Servidor local: Apache + PHP + MySQL através do XAMPP.
- Geração documental: PDF no servidor; impressão térmica para POS quando suportada.

2.2 Camadas

Presentation Layer: páginas, componentes UI, modais, formulários, tabelas, dashboard e POS.

Application Layer: controllers, services, validators, workflows e autorização.

Domain/Business Layer: regras de vendas, mesas, pedidos, stock, caixa, compras e faturação.

Data Layer: repositories/queries, transações, migrations/seeds e ligação MySQL.

Integration Layer: impressoras, QR/links, API, AGT quando aplicável, serviços externos e IA.

3. ESTRUTURA FUNCIONAL DO SISTEMA

Módulo	Funções principais
Dashboard	KPIs, vendas, pedidos, mesas, stock baixo, atividades, gráficos, resumo financeiro.
Mesas	Mapa de mesas, estados, zonas, ocupação, transferência, junção/divisão, reservas.
Pedidos / POS	Novo pedido, itens, modificadores, descontos, notas, divisão de conta, pagamento.
Cozinha	KDS, filas, prioridades, estados: recebido, em preparação, pronto, entregue/cancelado.
Clientes	Cadastro, histórico, contactos, consumo, crédito se habilitado.
Produtos / Menu	Pratos, bebidas, categorias, preços, variantes, complementos, combos.
Stock	Entradas, saídas, ajustes, inventário, lotes, validade, mínimos, movimentos.
Compras	Fornecedores, encomendas, receção, faturas de compra, custos e atualização de stock.
Fornecedores	Cadastro, contactos, documentos, histórico de compras e saldos.
Caixa	Abertura/fecho, sangrias, reforços, recebimentos, métodos de pagamento, conferência.
Faturação	Documentos de venda, séries, numeração, impressão/PDF, anulação/correção conforme regras.
Relatórios	Vendas, produtos, mesas, caixa, stock, compras, margens, utilizadores e auditoria.
RH	Utilizadores operacionais, turnos e funções; gestão laboral completa pode ser fase posterior.
Configurações	Empresa, filial, impostos, séries, métodos de pagamento, impressoras, permissões.
Auditoria	Login, alterações, cancelamentos, descontos, movimentos de caixa, stock e documentos.

4. DASHBOARD — REQUISITOS

O dashboard deverá reproduzir a lógica da referência fornecida, mas os valores deverão ser calculados a partir da base de dados, nunca gravados manualmente.

- Vendas de hoje: total, quantidade de transações e comparação com período anterior.
- Pedidos de hoje: pedidos recebidos/atendidos/cancelados.
- Clientes ativos: clientes com atividade no período configurado.
- Mesas ocupadas: ocupadas/total e percentagem.
- Stock baixo: quantidade de produtos abaixo do stock mínimo.
- Vendas por categoria: gráfico dinâmico.
- Pratos mais vendidos: ranking por quantidade e/ou valor.
- Últimas atividades: feed de eventos do sistema.
- Evolução de vendas: série temporal configurável.
- Desempenho do restaurante: ocupação, vendas, ticket médio e outros KPIs.
- Resumo financeiro: vendas, despesas registadas, recebimentos e indicadores.

Filtros mínimos: hoje, ontem, 7 dias, 30 dias, período personalizado e filial.

5. POS / PEDIDOS — FLUXO OPERACIONAL

- Selecionar mesa ou atendimento ao balcão.
- Adicionar produtos por categoria ou pesquisa.
- Permitir quantidade, observação, modificadores e complementos.

- Enviar itens para cozinha por estação de produção.
- Permitir alteração do pedido enquanto estiver no estado permitido.
- Permitir transferência de mesa e junção/divisão de mesas conforme permissões.
- Permitir divisão da conta por itens ou por valor.
- Aplicar desconto somente com permissão configurada.
- Selecionar método de pagamento e confirmar recebimento.
- Fechar pedido e gerar documento de venda/fatura conforme configuração.
- Registrar automaticamente movimentos de caixa e stock conforme as regras definidas.

6. GESTÃO DE MESAS

- Cadastro de mesas com código, capacidade, zona e estado.
- Estados mínimos: livre, ocupada, reservada, aguardando pagamento, bloqueada.
- Mapa visual configurável.
- Abertura de atendimento associada à mesa.
- Controlo de tempo de ocupação.
- Transferência de pedido entre mesas.
- Junção e separação de contas.
- Histórico de utilização por mesa.

7. COZINHA / KDS

A cozinha deve possuir uma interface própria, otimizada para operação rápida e sem necessidade de abrir o módulo administrativo.

- Fila de pedidos em tempo real.
- Separação por estação: cozinha, bar, sobremesas, etc.
- Estados: novo, aceite, em preparação, pronto, entregue e cancelado.
- Temporizador por pedido.
- Prioridade e marcação de urgência.
- Atualização automática do POS quando o pedido muda de estado.
- Histórico para auditoria e análise de tempos de preparação.

8. STOCK E FICHA TÉCNICA

O sistema deve suportar tanto produtos de venda direta como pratos produzidos através de ficha técnica.

- Produto: SKU/código, nome, categoria, unidade, preço, custo, stock mínimo e ativo/inativo.
- Ingrediente: unidade de compra, unidade de consumo e fator de conversão.
- Ficha técnica: prato → ingredientes → quantidades → custo teórico.
- Venda de prato pode gerar baixa automática dos ingredientes conforme receita configurada.
- Entradas por compra, saídas por consumo, perdas, quebras e ajustes.
- Inventário físico com conferência e motivo de ajuste.
- Alertas de stock mínimo.
- Lotes e validade devem existir quando necessários.

9. COMPRAS

- Pedido interno de compra.
- Encomenda ao fornecedor.
- Receção parcial ou total.
- Entrada automática em stock após receção confirmada.
- Registo do custo de aquisição.
- Documento do fornecedor associado à compra.
- Histórico e consulta por fornecedor/produto/período.

10. CAIXA E FINANCEIRO OPERACIONAL

- Abertura de caixa com valor inicial.
- Um ou mais caixas por filial/terminal.
- Entradas, saídas, reforços e sangrias.
- Métodos configuráveis: numerário, TPA/cartão, transferência, referência/outros.
- Fecho de caixa com valor esperado versus valor contado.
- Registo de diferenças e justificações.
- Relatório de caixa por operador, turno, terminal e período.
- Não permitir apagar movimentos financeiros; usar estorno/correção auditável.

11. FATURAÇÃO

A faturação deve ser implementada como módulo documental independente do POS, embora integrada com ele.

- Numeração sequencial por série e tipo de documento.
- Estados: rascunho, emitido, anulado/corrigido conforme regra aplicável.
- Documento associado ao cliente quando necessário.
- Linhas com produto/serviço, quantidade, preço, desconto, imposto e total.
- Cálculo de subtotal, descontos, impostos e total.
- PDF e impressão.
- QR Code e elementos fiscais somente quando definidos pela configuração e requisitos legais aplicáveis.
- Parâmetros fiscais não devem ser codificados diretamente no PHP; devem existir em configuração.
- Preparar estrutura para integração com AGT e/ou SAF-T conforme requisitos aplicáveis ao ambiente de produção.

12. BASE DE DADOS — MODELO INICIAL

Tabela	Finalidade
users	Utilizadores e credenciais
roles / permissions	Perfis e permissões granulares
companies / branches	Empresa e filiais
tables / table_zones	Mesas e zonas
customers	Clientes
suppliers	Fornecedores

Tabela	Finalidade
categories	Categorias
products	Produtos/pratos/bebidas
product_modifiers	Complementos e opções
recipes / recipe_items	Fichas técnicas
stock / stock_movements	Existências e movimentos
purchases / purchase_items	Compras
orders / order_items	Pedidos
kitchen_tickets / kitchen_items	Produção/cozinha
payments	Pagamentos
cash_registers / cash_movements	Caixas e movimentos
invoices / invoice_items	Documentos de venda
document_series	Séries e numeração
reservations	Reservas
expenses	Despesas operacionais
inventory / inventory_items	Inventários físicos
audit_logs	Auditoria
settings	Parâmetros do sistema

As relações, índices, chaves estrangeiras, constraints, tipos de dados, auditoria e transações deverão ser definidos no SQL final. Campos de dinheiro devem usar DECIMAL, nunca FLOAT para valores financeiros.

13. API INTERNA

O sistema deve separar endpoints de leitura e escrita e utilizar respostas JSON padronizadas para operações assíncronas.

```
Exemplos de rotas:
GET /api/dashboard/summary
GET /api/tables
POST /api/orders
POST /api/orders/{id}/send-kitchen
PATCH /api/orders/{id}/status
POST /api/payments
POST /api/cash/close
GET /api/stock/low
POST /api/invoices
```

14. SEGURANÇA

- Passwords armazenadas com password_hash/password_verify ou mecanismo equivalente.
- Sessões seguras, timeout e logout real.
- CSRF protection nos formulários.
- Validação server-side de todos os dados.
- Queries parametrizadas/PDO para impedir SQL Injection.
- Escaping de saída para reduzir XSS.
- RBAC com permissões por módulo e ação.
- Registo de login, logout, falhas, alterações, exclusões lógicas, descontos, anulações e operações críticas.
- Backups da base de dados e política de restauração.

15. PERFIS E PERMISSÕES

As permissões devem ser por ação, não apenas por menu.

- Administrador: configuração global e acesso autorizado a todas as áreas.
- Gerente: operação, relatórios, stock, compras e controlo financeiro conforme política.
- Caixa: POS, pagamentos e caixa.
- Garçom: mesas e pedidos; sem acesso a configurações sensíveis.
- Cozinha: KDS e atualização de produção.
- Stock/Compras: inventário, entradas, compras e fornecedores.
- Contabilidade/Fiscal: documentos, relatórios e integrações autorizadas.

Exemplos de permissões: view, create, edit, delete, cancel, approve, discount, close_cash, reopen_cash, export, print, configure.

16. UI/UX — REFERÊNCIA VISUAL

- Cabeçalho superior com marca, pesquisa global, notificações, ajuda, utilizador e logout.
- Menu lateral para acesso por módulos.
- Barra de ações rápidas no topo do conteúdo.
- Dashboard com cartões KPI, gráficos e listas.
- Menu inferior persistente para os módulos operacionais principais, conforme a referência visual.
- Paleta baseada em azul, branco e tons claros; estados de alerta devem possuir distinção visual.
- Interface responsiva e compatível com POS/tablet.
- Não usar dados fictícios como se fossem dados reais: a UI deve consumir dados da API/base de dados.

17. ASSISTENTE DE IA

O assistente de IA deve ser tratado como serviço complementar, nunca como fonte autónoma de dados financeiros.

- Responder a perguntas sobre utilização do sistema.
- Explicar erros e fluxos operacionais.
- Consultar dados autorizados através de funções controladas.
- Gerar resumos de vendas/stock quando autorizado.
- Respeitar as permissões do utilizador.
- Não executar operações críticas sem confirmação e autorização.

18. REGRAS DE INTEGRIDADE

- Operações que alteram stock + pedido + pagamento devem usar transações quando fizerem parte do mesmo fluxo.
- Pedido pago não deve ser eliminado fisicamente.
- Documento emitido não deve ser apagado; deve seguir o fluxo de anulação/correção permitido.
- Movimento de caixa não deve ser apagado.
- Cada movimento de stock deve possuir origem e utilizador.
- Toda alteração crítica deve manter created_at, updated_at e user_id quando aplicável.
- Soft delete deve ser usado onde a rastreabilidade exigir preservação histórica.

19. ESTRUTURA DE PASTAS RECOMENDADA

Exemplo de organização para PHP estruturado/MVC:

```
public/
  ■■■ index.php
  ■■■ assets/css/
  ■■■ assets/js/
  ■■■ assets/img/
app/
  ■■■ controllers/
  ■■■ models/
  ■■■ services/
  ■■■ repositories/
  ■■■ middleware/
  ■■■ validators/
  ■■■ helpers/
  ■■■ views/
config/
database/
  ■■■ migrations/
  ■■■ seeds/
  ■■■ restaurante.sql
routes/
storage/
  ■■■ logs/
  ■■■ documents/
api/
.env / config.php
README.md
```

20. FLUXO PRINCIPAL DE VENDA

1. Utilizador inicia sessão → 2. abre/seleciona caixa → 3. seleciona mesa → 4. cria pedido → 5. adiciona itens → 6. envia para cozinha → 7. cozinha produz → 8. itens são entregues → 9. pedido é fechado → 10. pagamento é registado → 11. documento é emitido → 12. caixa é atualizado → 13. stock/ingredientes são baixados → 14. auditoria regista a operação.

21. CRITÉRIOS DE ACEITAÇÃO

- Instalação limpa em XAMPP sem alterações manuais no código.
- Importação do SQL cria todas as tabelas, índices e dados mínimos necessários.
- Login, logout, sessão e permissões funcionam corretamente.
- Um pedido criado no POS aparece na cozinha sem duplicação.
- Alterações de estado da cozinha refletem no atendimento.
- Pagamento atualiza caixa e fecha o pedido conforme regra.
- Venda de prato com ficha técnica atualiza stock/ingredientes.
- Relatórios utilizam dados reais e filtros funcionais.
- Documentos possuem numeração consistente e não permitem duplicação.
- Operações críticas ficam disponíveis no audit log.
- Não existem botões visíveis que não executem uma função definida.
- Erros devem apresentar mensagens compreensíveis e ser registados no log técnico.

22. FASES DE DESENVOLVIMENTO

Fase	Entrega
1	Arquitetura, base de dados, autenticação e layout base

Fase	Entrega
2	Dashboard, configurações, utilizadores e permissões
3	Mesas + POS + pedidos
4	Cozinha/KDS
5	Produtos + fichas técnicas + stock
6	Compras + fornecedores + inventário
7	Caixa + pagamentos + faturação
8	Relatórios + auditoria + exportações
9	Integrações, impressões, QR e requisitos fiscais aplicáveis
10	Testes, segurança, otimização, backup e entrega final

23. ENTREGÁVEIS OBRIGATÓRIOS DO PROGRAMADOR

- Código-fonte completo e organizado.
- Base de dados SQL completa.
- Script de instalação/configuração para XAMPP.
- Ficheiro de configuração de ambiente.
- Manual técnico de instalação.
- Manual de utilização dos módulos.
- Documentação da API.
- Diagrama ER da base de dados.
- Lista de permissões e perfis.
- Lista de endpoints.
- Plano de backup e restauração.
- Credenciais iniciais de teste documentadas e alteração obrigatória no primeiro acesso.

24. OBSERVAÇÃO PARA O PROGRAMADOR

Não desenvolver apenas o desenho apresentado. A imagem fornecida é a referência de UI/UX. O objetivo é implementar um ERP/POS funcional, com base de dados relacional, regras de negócio, transações, permissões, auditoria e integração entre módulos. Cada botão, cartão, gráfico, tabela e indicador deve estar ligado a uma função real ou ser removido da interface. Não utilizar valores fixos para simular funcionalidades em produção.

FIM DO DOCUMENTO — RestaurantePro v1.0